

UNIVERSITY
OF SUSSEX

University of Sussex
School of Mathematical and Physical Sciences

A Small, Reproducible QAOA Benchmark in Bloch: Max-Cut on an 8-Node Cycle

Candidate: Akshay Pal
Candidate Number: 288756

Supervisor: *Professor Barry Garraway*

Contents

1 Abstract **1**

2 Introduction and background **1**

3 Problem definition **1**

3.1 The C_8 graph and it's optimum 1

3.2 Spin representation and cost Hamiltonian 2

4 Research methodology and implementation **2**

4.1 QAOA definition (depth $p = 1$) 3

4.2 Shot-based evaluation and empirical objective 3

4.3 Code repository 4

5 Bloch vs Qiskit: toolchain comparison **4**

6 Analysis and discussion **4**

6.1 Candidate distributional tests 4

6.2 Chosen primary metric: distance from optimum 5

6.3 Results 5

6.4 What the numbers mean: validity claims supported quantitatively 6

7 Conclusion **7**

1 Abstract

Variational quantum algorithms (VQAs) are widely used in the noisy intermediate-scale quantum era, yet their empirical behaviour can depend strongly on toolchain conventions and evaluation policies, complicating reproducibility and correctness validation. This dissertation validates Bloch, a standalone strongly-typed quantum programming language, on a canonical variational workflow by checking that it reproduces the expected solution structure for Max-Cut on the 8-node cycle graph C_8 under the Quantum Approximate Optimisation Algorithm (QAOA). Qiskit is used as a reference implementation because it is an established and widely adopted framework. A matched depth QAOA ansatz is implemented in both toolchains, circuits are evaluated via projective measurement with a fixed budget of 8192 shots per objective evaluation, and parameters are optimised using parameter-shift gradients with gradient *ascent*. Quantitative validation is reported using the empirical expected cut value, the probability of sampling an optimal cut, and associated optimality gaps. Candidate distributional tests (total variation distance and Kolmogorov-Smirnov) are defined and critically assessed but are not used as primary criteria under finite-shot sampling and RNG asymmetry. The measured outcomes concentrate on the two optimal C_8 cuts and yield expected cut values close to the reference, supporting Bloch as a valid toolchain for quantum algorithm development.

2 Introduction and background

The noisy intermediate-scale quantum (NISQ) era is characterised by limited coherence times and imperfect gates, restricting feasible circuit depth and motivating hybrid quantum-classical approaches [1]. Variational quantum algorithms (VQAs) address these constraints by repeatedly executing a parameterised quantum circuit and using a classical optimiser to update parameters based on measured outcomes. Canonical examples include the Variational Quantum Eigensolver (VQE) [2] and the Quantum Approximate Optimisation Algorithm (QAOA) [3, 4]. However, VQAs are sensitive to design and implementation choices (ansatz conventions, shot budgets, optimiser configuration, simulator defaults), and this toolchain coupling complicates reproducibility and correctness validation [5].

Bloch is a standalone, strongly-typed, hardware-agnostic quantum programming language [6]. A natural question is whether Bloch behaves correctly on standard variational workloads relative to established frameworks. The primary aim of this dissertation is validation: demonstrate that Bloch produces the correct solution structure on a small analytically tractable QAOA instance. Comparison to Qiskit is secondary and is used as a reference check rather than a performance contest.

The remainder of this dissertation is structured as follows. Section 3 formalises Max-Cut on C_8 and derives the known optimum used as ground truth. Section 4 defines the QAOA ansatz, measurement-based objective estimation, and the parameter-shift gradient ascent routine used for optimisation, together with reproducibility controls and code provenance. Section 5 summarises relevant toolchain differences between Bloch and Qiskit that affect validation. Section 6 presents quantitative evaluation and interprets the results in terms of Bloch's validity on this benchmark, including a discussion of alternative statistical tests. Section 7 concludes and outlines future directions.

3 Problem definition

3.1 The C_8 graph and it's optimum

Let $G = (V, E)$ be a simple undirected graph with $|V| = n$ and $E \subseteq \{\{i, j\} : i \neq j\}$. A cut is encoded by a bitstring $x \in \{0, 1\}^n$, where x_i indicates the partition membership of vertex i. An edge $\{i, j\}$ is cut if $x_i \neq x_j$. The cut value is

$$C(x) = \sum_{\{i,j\} \in E} \mathbb{I}\{x_i \neq x_j\} = \sum_{\{i,j\} \in E} (x_i \oplus x_j), \tag{1}$$

where $\mathbb{I}$ is the identity matrix and $\oplus$ represents XOR. The Max-Cut optimum is then given by

$$C^* = \mathrm{MaxCut}(G) = \max_{x \in \{0,1\}^n} C(x). \tag{2}$$

Following the 1-based convention used in [7], we can define the following vertices and edges for the C_8 graph,

$$V = \{1, 2, 3, 4, 5, 6, 7, 8\} \tag{3}$$

$$E = \big\{\{i, i+1\} : i = 1, \ldots, 7\big\} \cup \big\{\{8, 1\}\big\} = \{\{1, 2\}, \{2, 3\}, \ldots, \{8, 1\}\} \tag{4}$$

A graph is bipartite if vertices can be partitioned into two sets such that every edge connects across the partition. For cycle graphs, C_n is bipartite if and only if n is even [7]. Since 8 is even, C_8 is bipartite. The alternating assignment

$$x_i = i \bmod 2$$

cuts every edge in E (each edge joins an odd vertex to an even vertex), hence $C(x) = |E| = 8$. No cut can exceed $|E|$, so

$$C^* = \mathrm{MaxCut}(C_8) = |E| = 8. \tag{5}$$

There are exactly two optimal assignments (the two alternating patterns), corresponding to bitstrings 01010101 and 10101010 under a fixed vertex ordering.

3.2 Spin representation and cost Hamiltonian

QAOA is naturally expressed using observables with eigenvalues ± 1. To align the cut objective with Pauli-Z eigenvalues, map $x_i \in \{0, 1\}$ to spins $z_i \in \{-1, +1\}$ via

$$z_i = 1 - 2x_i \quad \Longleftrightarrow \quad x_i = \frac{1 - z_i}{2}. \tag{6}$$

Then $\mathbb{I}\{x_i \neq x_j\} = (1 - z_i z_j)/2$. In the quantum setting, computational basis states are eigenstates of Pauli-Z with eigenvalues ± 1, so Z_i acts as the operator analogue of z_i. The Max-Cut cost Hamiltonian is

$$H_C = \sum_{\{i,j\} \in E} \frac{1 - Z_i Z_j}{2} = \sum_{i=1}^{7} \frac{1 - Z_i Z_{i+1}}{2} + \frac{1 - Z_8 Z_1}{2}. \tag{7}$$

4 Research methodology and implementation

Hybrid variational workflow

Figure 1 summarises the VQA loop used in this benchmark: the circuit prepares a trial state, finite shots estimate the objective, and a classical optimiser updates parameters. Finite-shot measurement is a source of stochasticity even under noiseless state evolution.

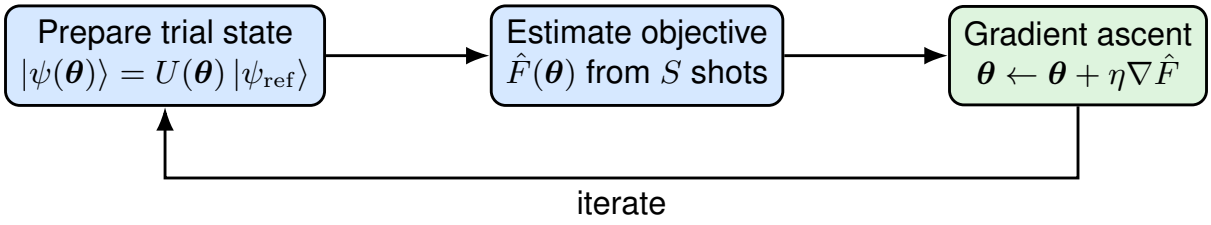


Figure 1: Hybrid VQA loop used by QAOA. The objective is estimated from finite shots, producing a Monte Carlo estimator even under noiseless evolution.

4.1 QAOA definition (depth $p = 1$)

The mathematical graph uses vertex labels $1, \ldots, 8$ (Section 3). In Bloch and Qiskit, qubits are indexed $0, \ldots, 7$. The implementation therefore uses the mapping

$$\text{vertex } i \longleftrightarrow \text{qubit } (i-1), \tag{8}$$

so the edge $\{i, j\}$ in the mathematical graph is realised as operations on qubits $(i-1, j-1)$ in code. This mapping is purely notational; it preserves adjacency and therefore preserves the objective.

QAOA alternates a cost unitary derived from H_C with a mixer unitary [3, 4]. For $p = 1$, the parameterised state is

$$|\psi(\gamma, \beta)\rangle = U_M(\beta)\, U_C(\gamma)\, |+\rangle^{\otimes 8}, \tag{9}$$

where $|+\rangle^{\otimes 8}$ is prepared by applying Hadamard gates to $|0\rangle^{\otimes 8}$. The cost unitary is

$$U_C(\gamma) = e^{-i\gamma H_C}, \tag{10}$$

with H_C defined in (7). The mixer Hamiltonian is

$$H_M = \sum_{i=1}^{8} X_i, \qquad U_M(\beta) = e^{-i\beta H_M}. \tag{11}$$

Since $R_X(\theta) = e^{-i\theta X/2}$, applying $R_X(2\beta)$ to each qubit implements $e^{-i\beta X}$ per qubit, matching (11).

Gate-level circuit realisation

The cost layer is implemented edge-by-edge using the standard decomposition of a ZZ rotation: for each edge $\{i, j\} \in E$, the code applies

$$\mathrm{CX}(i-1, j-1) \to \mathrm{RZ}_{j-1}(-\gamma) \to \mathrm{CX}(i-1, j-1), \tag{12}$$

followed by mixer rotations $\mathrm{RX}(2\beta)$ on all eight qubits, then measurement in the computational basis. This gate pattern is identical across Bloch and Qiskit under the vertex-qubit mapping.

4.2 Shot-based evaluation and empirical objective

Even under ideal state evolution, projective measurement is probabilistic. For fixed (γ, β), measuring $|\psi(\gamma, \beta)\rangle$ yields a random bitstring $X \in \{0, 1\}^8$ with distribution $p_{\gamma,\beta}(x)$. With S shots, observed counts define the empirical distribution

$$\hat{p}_{\gamma,\beta}(x) = \frac{\mathrm{counts}(x)}{S}.$$

The empirical objective is the expected cut value under the empirical distribution:

$$\hat{F}(\theta) = \hat{F}(\gamma, \beta) \equiv \widehat{\mathbb{E}}_{x \sim \hat{p}_{\gamma,\beta}}[C(x)] = \sum_{x \in \{0,1\}^8} \hat{p}_{\gamma,\beta}(x)\, C(x). \tag{13}$$

This dissertation fixes $S = 8192$ for every objective evaluation in both toolchains.

Parameter-shift gradient *ascent*

Parameter-shift gradients provide exact derivatives for suitable rotation gates [8]:

$$\frac{\partial}{\partial\theta} F(\theta) = \frac{1}{2}\big(F(\theta + \tfrac{\pi}{2}) - F(\theta - \tfrac{\pi}{2})\big). \tag{14}$$

The optimiser uses *gradient ascent* because the objective is to maximise expected cut value. Each iteration evaluates $\hat{F}$ at five points:

$$(\gamma, \beta),\ (\gamma \pm \tfrac{\pi}{2}, \beta),\ (\gamma, \beta \pm \tfrac{\pi}{2}), \tag{15}$$

and computes

$$\partial_\gamma \hat{F} = \frac{1}{2}\Big[\hat{F}\Big(\gamma + \frac{\pi}{2}, \beta\Big) - \hat{F}\Big(\gamma - \frac{\pi}{2}, \beta\Big)\Big], \tag{16}$$

$$\partial_\beta \hat{F} = \frac{1}{2}\Big[\hat{F}\Big(\gamma, \beta + \frac{\pi}{2}\Big) - \hat{F}\Big(\gamma, \beta - \frac{\pi}{2}\Big)\Big]. \tag{17}$$

Parameters are updated by

$$\gamma \leftarrow \gamma + \eta \partial_\gamma \hat{F} \tag{18}$$

$$\beta \leftarrow \beta + \eta \partial_\beta \hat{F} \tag{19}$$

with $\eta = 0.12$, $(\gamma_0, \beta_0) = (0.7, 0.3)$, and 25 iterations. Each iteration uses 5 circuit evaluations, so $25 \times 5 = 125$ objective evaluations per toolchain, followed by a final evaluation at (γ^*, β^*).

4.3 Code repository

All code used to generate results (experiment runner, Bloch and Qiskit implementations, and reproduction instructions) is available at:

`https://github.com/akshaypal123/qtam-msc-project`.

5 Bloch vs Qiskit: toolchain comparison

Qiskit is a mature Python ecosystem with extensive tooling, simulators, and deep integration into IBM Quantum backends [9]. It's strengths include feature completeness, established conventions, and a large user base. A drawback for reproducibility-heavy work is Python environment fragility (dependency drift) and the presence of many configuration "degrees of freedom" (transpiler settings, backend options) that can change behaviour between environments.

Bloch is a standalone language and toolchain [6]. Strong typing reduces ambiguity in hybrid quantum-classical programs, and backend agnosticity aims to separate program semantics from any single provider. As a newer system, Bloch is less feature-rich and less mature; in this benchmark it exposes sampling via a CLI interface and does not expose internal statevectors or explicit RNG seeding for measurement sampling. As stated in the introduction, Qiskit is treated as the reference implementation and the goal is validation of Bloch's correctness on this benchmark rather than outperforming Qiskit.

6 Analysis and discussion

This section reports quantitative results and interprets what they mean for toolchain validity. Two evaluation questions are relevant: (i) how close is the output to the known optimum $C^* = 8$ and the two optimal bitstrings, and (ii) how similar are Bloch's empirical results to the Qiskit reference under finite-shot sampling.

6.1 Candidate distributional tests

Total variation distance (TVD). For empirical distributions $\hat{p}$ and $\hat{q}$ on $\{0,1\}^8$,

$$\mathrm{TVD}(\hat{p}, \hat{q}) = \frac{1}{2} \sum_{x \in \{0,1\}^8} |\hat{p}(x) - \hat{q}(x)|\,.$$

Pros: order-invariant and natural for categorical outcomes. *Cons:* strongly influenced by finite-shot variance and, in this benchmark, RNG asymmetry (seeded Qiskit sampling versus non-seeded Bloch CLI sampling). Therefore TVD can conflate sampling artefacts with semantic differences, and is not used as the primary success criterion.

Kolmogorov-Smirnov (KS). For one-dimensional variables with CDFs F and G,

$$D_{KS} = \sup_t |F(t) - G(t)|.$$

KS requires an ordering to define a CDF. The raw outcomes here are categorical bitstrings with no canonical ordering; any imposed ordering (e.g., integer encoding) is arbitrary. KS could be applied to a derived scalar such as $C(x) \in \{0, \ldots, 8\}$, but then direct discrete-bin comparisons are more appropriate. KS is therefore not used.

6.2 Chosen primary metric: distance from optimum

Because $C^* = 8$ is known exactly, validation focuses on distance-from-optimum metrics derived from samples:

$$\hat{\Delta} = 8 - \hat{F}(\gamma, \beta),$$

and the empirical success probability of sampling an optimal cut:

$$\hat{P}_{\text{success}} = \sum_{x \in S^*} \hat{p}(x), \qquad S^* = \{01010101, 10101010\}.$$

These metrics are meaningful under Bloch's sampling interface and directly quantify correctness relative to ground truth.

6.3 Results

Table 1 reports optimised parameters and headline metrics for one representative run.

Table 1: Summary metrics (one run): QAOA on C_8 at $p = 1$, 25 optimisation iterations, 8192 shots per evaluation.

Toolchain	γ^*	β^*	$\hat{F} = \widehat{\mathbb{E}}[C]$	$\hat{\Delta} = 8 - \hat{F}$	$\hat{P}_{\text{success}}$	Final eval (s)
Bloch	0.702036	0.300483	5.843506	2.156494	0.108154	0.985
Qiskit	0.693218	0.292603	5.793213	2.206787	0.104004	0.010

The most frequent sampled outputs are the two optimal cuts in both toolchains (Table 2).

Table 2: Top outcomes from final evaluation (8192 shots).

Toolchain	Bitstring	Count	$\hat{p}(x)$	Cut value $C(x)$
Bloch	10101010	448	0.054688	8
Bloch	01010101	438	0.053467	8
Bloch	10101100	151	0.018433	6
Bloch	10110010	150	0.018311	6
Qiskit	01010101	433	0.052856	8
Qiskit	10101010	419	0.051147	8
Qiskit	10010101	145	0.017700	6
Qiskit	10110010	145	0.017700	6

6.4 What the numbers mean: validity claims supported quantitatively

The ground truth for this benchmark is (i) the optimum cut value $C^* = 8$ and (ii) the fact that exactly two alternating assignments achieve it, corresponding to the bitstrings $S^* = \{01010101, 10101010\}$ under the fixed vertex ordering.

Optimal-solution mass. Define the empirical success probability

$$\hat{P}_{\text{success}} = \sum_{x \in S^*} \hat{p}(x) = \frac{N^*}{S},$$

where N^* is the number of observed shots that land in S^* and $S = 8192$ is the shot budget. In the final evaluation, Bloch obtains $\hat{P}_{\text{success}} = 0.108154$ and Qiskit obtains $\hat{P}_{\text{success}} = 0.104004$, with both optimal bitstrings appearing as the top two outcomes. This supports the claim that Bloch reproduces the correct *solution structure* of C_8 under QAOA, namely that probability mass concentrates on the two alternating maximum cuts.

Expected cut value and optimality gap. The primary performance metric is the empirical expected cut value

$$\hat{F} = \widehat{\mathbb{E}}[C] = \sum_{x \in \{0,1\}^8} \hat{p}(x)\, C(x) = \frac{1}{S} \sum_{k=1}^{S} C(X_k),$$

which is a Monte Carlo estimator of $\mathbb{E}[C]$ under the circuit output distribution. The corresponding empirical optimality gap is

$$\hat{\Delta} = C^* - \hat{F} = 8 - \hat{F}.$$

In this run, $\hat{F}_{\text{Bloch}} = 5.843506$ and $\hat{F}_{\text{Qiskit}} = 5.793213$, giving

$$\hat{\Delta}_{\text{Bloch}} = 2.156494, \qquad \hat{\Delta}_{\text{Qiskit}} = 2.206787,$$

and an absolute toolchain difference

$$\left| \hat{F}_{\text{Bloch}} - \hat{F}_{\text{Qiskit}} \right| \approx 0.050293.$$

Finite-shot uncertainty: conservative bound Because each shot produces a cut value $C(X_k) \in \{0, 1, \ldots, 8\}$, it is bounded in $[0, 8]$. For any random variable $Y \in [a, b]$, the variance satisfies $\mathrm{Var}(Y) \leq (b-a)^2/4$. Applying this with $a = 0$ and $b = 8$ gives $\mathrm{Var}(C) \leq 16$ and hence $\mathrm{SD}(C) \leq 4$. Assuming independent shots (a standard model for repeated projective measurement),

$$\mathrm{Var}(\hat{F}) = \frac{\mathrm{Var}(C)}{S} \leq \frac{16}{S}, \qquad \mathrm{SD}(\hat{F}) = \sqrt{\mathrm{Var}(\hat{F})} \leq \frac{4}{\sqrt{S}}.$$

For $S = 8192$ this yields the worst-case bound

$$\mathrm{SD}(\hat{F}) \leq \frac{4}{\sqrt{8192}} \approx 0.0442.$$

Therefore, a single-run difference of ≈ 0.0503 between Bloch and Qiskit expected-cut estimates is plausibly explainable by finite-shot effects (and in this benchmark, additional RNG asymmetry: Qiskit sampling is explicitly seeded, while Bloch sampling via the CLI is not).

Interpretation as a validation result Taken together, (i) concentration of probability mass on the two known optimal cuts, (ii) similar $\hat{P}_{\text{success}}$ values, and (iii) similar expected cut values within conservative finite-shot uncertainty provide quantitative evidence that Bloch is *valid* for this canonical QAOA Max-Cut benchmark when compared against a Qiskit reference implementation under matched circuit structure, shot budgets, and optimisation protocol.

7 Conclusion

This dissertation validated Bloch on a canonical variational benchmark by implementing depth-$p = 1$ QAOA for Max-Cut on C_8 under a matched experimental protocol and comparing to a Qiskit reference. The known optimum $C^* = 8$ and the two optimal alternating solutions provide ground truth for correctness checks. Bloch places the largest observed probability mass on the two optimal bitstrings and achieves an expected cut value close to the reference under an identical shot budget and optimiser configuration. These quantitative results support the conclusion that Bloch is a valid toolchain for small reproducible QAOA benchmarks.

This benchmark is intentionally narrow: a single small instance (C_8), fixed depth ($p = 1$), and ideal simulation. The validation claim is therefore local: Bloch behaves correctly on this canonical QAOA benchmark under a matched protocol. Stronger claims would require a suite of instances (e.g., multiple cycles and small regular graphs), higher depth p, and explicit noise models. It is also an explicit product goal that Bloch improves on established frameworks in *developer experience and usability* (reproducible project structure, strong typing, backend-agnostic execution), rather than claiming raw simulator speedups over heavily optimised Python toolchains.

Future work should expand validation beyond a single instance by introducing a small benchmark suite and deeper QAOA circuits, and by adding noise models to assess robustness. Over time, Bloch's intended "outperformance" over Qiskit should be understood primarily as improved usability and reproducibility for hybrid quantum software engineering, rather than a claim about raw execution speed in ideal simulators.

References

[1] John Preskill. "Quantum Computing in the NISQ era and beyond". In: *Quantum* 2 (2018), p. 79. DOI: 10.22331/q-2018-08-06-79. URL: https://quantum-journal.org/papers/q-2018-08-06-79/.

[2] Alberto Peruzzo et al. "A variational eigenvalue solver on a photonic quantum processor". In: *Nature Communications* 5 (2014), p. 4213. DOI: 10.1038/ncomms5213. URL: https://doi.org/10.1038/ncomms5213.

[3] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. "A Quantum Approximate Optimization Algorithm". In: (2014). arXiv: 1411.4028 [quant-ph]. URL: https://arxiv.org/abs/1411.4028.

[4] Stuart Hadfield et al. "From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz". In: *Algorithms* 12.2 (2019), p. 34. DOI: 10.3390/a12020034.

[5] Jarrod R. McClean et al. "Barren Plateaus in Quantum Neural Network Training Landscapes". In: *Nature Communications* 9 (2018), p. 4812. DOI: 10.1038/s41467-018-07090-4.

[6] Akshay Pal. "Bloch: a strongly typed, hardware-agnostic, hybrid quantum programming language". In: *Journal of Open Source Software* 11.117 (2026), p. 9625. DOI: 10.21105/joss.09625. URL: https://joss.theoj.org/papers/10.21105/joss.09625.

[7] Richard J. Trudeau. *Introduction to Graph Theory*. Stanford Inversiones SpA, Jan. 2017. ISBN: 978-1-68411-231-9.

[8] Maria Schuld et al. "Evaluating analytic gradients on quantum hardware". In: *Physical Review A* 99.3 (2019), p. 032331. DOI: 10.1103/PhysRevA.99.032331. URL: https://link.aps.org/doi/10.1103/PhysRevA.99.032331.

[9] Ali Javadi-Abhari et al. *Quantum computing with Qiskit*. 2024. DOI: 10.48550/arXiv.2405.08810. arXiv: 2405.08810 [quant-ph].